

SABEKA BACKEND

Architecture & Modernization Report

Generated on April 29, 2026

REPORT 1 Clean System Design

REPORT 2 Modular Refactor

REPORT 3 DB Modernization

8

Business Domains

5

Migration Phases

7

Architecture Sprints

3

Layer Patterns

This document consolidates three architectural analysis reports covering the full Sabeka backend system: a clean system design blueprint, a modular refactor implementation plan, and a MySQL-to-PostgreSQL database modernization strategy.

TABLE OF CONTENTS

PART 1 **Clean System Design**

- 1.1 Architectural Style & Principles
- 1.2 System Context & Logical Components
- 1.3 Layer Responsibilities
- 1.4 Transaction & Consistency Design
- 1.5 API Design Standards
- 1.6 Security Architecture
- 1.7 Caching, Async Processing & Observability
- 1.8 Deployment & Quality Engineering
- 1.9 Migration Plan

PART 2 **Modular Refactor Implementation**

- 2.1 Target Architecture Overview
- 2.2 Transaction Architecture (Unit of Work)
- 2.3 Flow Diagrams
- 2.4 Sample Code Templates
- 2.5 Domain Migration Checklist
- 2.6 Team Backlog Cards

PART 3 **Database Modernization — MySQL to PostgreSQL**

- 3.1 Observations & MySQL-Specific Patterns
- 3.2 PostgreSQL Design Standards
- 3.3 Type Mapping Strategy
- 3.4 Financial Integrity Design
- 3.5 Table-Class Refactor Recommendations
- 3.6 Migration Architecture (6 Phases)
- 3.7 Tooling & Application Layer Changes

APPENDIX **Risk Register & Definition of Done**

PART 1
CLEAN SYSTEM DESIGN

1.1 Architectural Style & Principles

Sabeka adopts a **domain-oriented modular monolith** as the primary architecture. This balances delivery speed with strong internal boundaries, enabling future extraction of high-load modules into independent services when growth demands it.

Style	Domain-based modular monolith
Internal Pattern	Layered per domain: Controller → Service → Repository
Cross-cutting Kernel	Validation, errors, transactions, authz, logging, config
Scalability Path	Gradual extraction of high-load modules to microservices

Core Design Principles

Explicit Boundaries	Domains own their code, data models, and contracts
Fail-Fast Validation	Reject invalid input at the API boundary, not deep in logic
Transactional Consistency	Multi-table writes always run in one atomic transaction
Default-Deny Authorization	No access unless explicitly permitted
Idempotent Interactions	External calls are safe to retry without side effects
Observable by Default	Structured logs, metrics, and traces on every critical path
Backward-Compatible Steps	DB and code migrations deploy without downtime

1.2 System Context & Logical Components

#	Component	Responsibility
1	API Gateway Layer	Express routes + middleware (auth, rate-limit, validation)
2	Domain Modules	Transaction, Wallet, Order, User, Auth, Inventory, Payment
3	Data Access Layer	Knex/Objection repositories — SQL only, no business logic
4	Transaction Manager	UnitOfWork — BEGIN / COMMIT / ROLLBACK abstraction
5	Integration Layer	Payment providers, messaging, third-party APIs
6	Cache / Session Layer	Redis for sessions, rate-limit, short-lived data
7	Observability Layer	Structured logs, Prometheus metrics, distributed traces
8	Async Processing	Queue workers + scheduled jobs (notifications, reconciliation)

1.3 Layer Responsibilities

Controller	■ HTTP mapping only — parse request, read validated DTO ■ Call one service method ■ Map result to HTTP response shape ■ Never call DB directly, never enforce business rules
Service	■ Own the business use case end-to-end ■ Define and own the transaction boundary ■ Enforce domain invariants and business rules ■ Coordinate repositories and integrations ■ Throw typed <code>AppError</code> subclasses on failure
Repository	■ DB query/mutation only — SQL or ORM calls ■ Reusable data access primitives ■ Accept optional transaction context (<code>tx</code>) ■ No HTTP handling, no authorization policy

1.4 Transaction & Consistency Design

Every critical multi-table write must execute under a single **UnitOfWork** transaction. The service layer defines boundaries; repositories receive the `tx` object to participate in the transaction.

Atomicity Safeguards Checklist

■	Wallet + ledger writes run in the same transaction
■	Order + inventory state writes run in the same transaction
■	Row locking (SELECT FOR UPDATE) on race-sensitive records
■	DB constraints act as the final integrity guard
■	Rollback on any exception path — no partial persistence
■	No external irreversible side effects before COMMIT
■	Idempotency key on retry-able write endpoints
■	Bounded retry policy for deadlock / serialization errors

Outbox Pattern for Post-Commit Side Effects

Step 1	Save business data + outbox event row in same DB transaction
Step 2	COMMIT once — both data and event are durable
Step 3	Background worker reads unpublished outbox rows
Step 4	Worker publishes event to queue/integration, marks row as processed

1.5 API Design Standards

Conventions

- Versioned prefix /api/v1
- Resource-based RESTful URLs
- Predictable pagination/filtering
- Idempotent semantics where appropriate

Request / Response Shape

- Success: { success: true, data, meta }
- Error: { success: false, error.code, error.message }
- requestId on every response
- Schema-first validation — service gets typed DTO

1.6 Security Architecture

Domain	Approach
Authentication	JWT access token with explicit expiry; session state in Redis for revocation
Authorization	Default-deny policy; centralized permission resolver; missing record = deny
Data Protection	Secrets via env/secret manager; PII redacted in logs; strict input limits
Rate Limiting	Endpoint-specific rate-limiting profiles; secure HTTP headers
Webhook Security	Signature verification; timestamp tolerance window; idempotency key

1.7 Caching, Async Processing & Observability

Area	Details
Structured Logging	JSON logs with timestamp, level, requestId, route, userId, durationMs; PII redacted
Metrics to Track	Request latency, error rate, TX commit/rollback counts, deadlock/retry rates
Tracing	Distributed traces for critical flows; error grouping and release tracking
Alerting	Rollback spikes, 5xx anomalies, queue depth lag, auth failure rate
Caching (Redis)	Route-level and entity-level keys; short/medium TTL; explicit invalidation on writes
Async Workers	Queue-backed: notifications, reports, external retries, reconciliation jobs
Job Reliability	Retries with exponential backoff; dead-letter queue; idempotent processors

1.8 Deployment & Quality Engineering

Runtime Topology

- Horizontal API nodes
- Redis cluster/instance
- Queue broker + worker nodes
- Monitoring stack
- Dev / staging / production parity
- Feature flags for phased rollouts

Test Pyramid

- Unit tests — service logic & edge cases
- Integration tests — DB transactions, rollback
- E2E tests — critical user journeys
- Concurrency race tests on balances/orders
- Authorization deny-by-default scenarios
- Import boundary lint checks in CI

1.9 Domain Migration Plan

Phase	Name	Key Activities	Output
Phase 0	Foundation	Shared modules: unitOfWork, validate, errorHandler, requestId, logger, config	<i>DI container + architecture lint rules in CI</i>
Phase 1	Financial Core	Migrate transaction domain (reference slice); wallet domain; idempotency + row locks	<i>Rollback & concurrency test suite</i>
Phase 2	Order & Inventory	Service-managed order flows; atomic inventory state transitions; outbox events	<i>Post-commit side effect safety</i>
Phase 3	User & Auth	Normalize auth/session services; default-deny authorization; token expiry/rotation	<i>Secure auth lifecycle tests</i>
Phase 4	Integrations & Workers	Move heavy/slow tasks to workers; retry + dead-letter; harden webhook verification	<i>Reliable async processing</i>
Phase 5	Hardening & Cleanup	Remove legacy mixed-layer code; baseline SLO dashboards; full regression + load test	<i>Production sign-off</i>

PART 2
MODULAR REFACTOR IMPLEMENTATION

2.1 Target Architecture Overview

The refactor reorganises the codebase into **domain-oriented vertical slices**, each containing its own Controller, Service, Repository, Validation, and Type definitions. A shared kernel handles cross-cutting concerns.

bootstrap/	app.ts, server.ts, routes.ts, container.ts
shared/db/	knex.ts, unitOfWork.ts, transactionContext.ts
shared/errors/	AppError.ts, HttpErrors.ts, errorMapper.ts
shared/middleware/	asyncHandler.ts, validate.ts, errorHandler.ts, requestContext.ts
shared/logger/	logger.ts, redact.ts
domains/transaction/	routes · controller · service · repository · validation · types
domains/wallet/	routes · controller · service · repository · validation
domains/order/	routes · controller · service · repository · validation
domains/user/	routes · controller · service · repository · validation
domains/auth/	routes · controller · service · repository · validation
workers/	jobs/, processors/
db/	migrations/, seeds/
tests/	unit/, integration/, e2e/

2.2 Transaction Architecture — Unit of Work

Six Rules to Prevent Partial Updates

1	All multi-entity writes execute inside one transaction
2	Repositories in critical flows must receive the tx context object
3	No external side-effects (email/webhook/event publish) before COMMIT
4	Row locking (FOR UPDATE) on balance and inventory sensitive records
5	Idempotency keys make retry-safe endpoints
6	DB constraints serve as the final integrity guard

2.3 Request Flow Diagrams

Success Flow

POST /api/transactions/debit — Happy Path
→ Client → Route → Validation Middleware → Controller.debit(dto)

→ Controller → Service.execute(dto, context)
→ Service → UnitOfWork.runInTransaction(work) → DB: BEGIN
→ Service → WalletRepo.findByIdForUpdate(tx) → DB: SELECT ... FOR UPDATE
→ Service → WalletRepo.decrementBalance(tx) + TransactionRepo.insertDebitEntry(tx)
→ Service → TransactionRepo.saveIdempotencyKey(tx) → DB: INSERT outbox row
→ UnitOfWork → DB: COMMIT → Controller returns 200 success JSON

Failure & Rollback Flow

POST /api/transactions/debit — Error Path
→ Client → Controller → Service → UnitOfWork → DB: BEGIN
→ Service → Repo: UPDATE wallets ... (ok)
→ Service → Repo: INSERT ledger ... (ok)
→ Service → Repo: UPDATE order status → DB: ERROR (constraint / deadlock)
→ Repository throws error → Service propagates → UnitOfWork → DB: ROLLBACK
→ UnitOfWork throws mapped AppError → Controller → ErrorHandler middleware
→ Client receives standardized error JSON (4xx/5xx) — zero partial persistence

Idempotent Webhook / Retry Flow

Webhook(eventId, payload) — Idempotency Guard
→ Provider → Webhook Controller → Service.processWebhook(dto)
→ Service → UnitOfWork → DB: BEGIN
→ Service → Repo: SELECT idempotency WHERE event_id = ? FOR UPDATE
→ If already processed → COMMIT, return existing result → 200 OK (no-op)
→ If new event → apply business writes + INSERT idempotency record → COMMIT
→ Provider receives 200 OK — safe to retry without double-processing

2.4 Sample Code Templates

UnitOfWork (shared/db/unitOfWork.ts)	runInTransaction(work): wraps a callback in a Knex transaction, auto-commits on success, auto-rolls back on any error.
Validation Middleware (shared/middleware/validate.ts)	Zod schema-based middleware; attaches typed 'req.validated' on success, throws ValidationError on failure.
Error Handler (shared/middleware/errorHandler.ts)	Global Express error middleware; maps AppError to structured JSON; hides internals for 500s.
Controller Template (domains/transaction/transaction.controller.ts)	Reads req.validated.body, calls transactionService.debit(dto), returns 200 JSON. No logic.
Service Template (domains/transaction/transaction.service.ts)	Uses uow.runInTransaction; checks idempotency key; loads wallet with FOR UPDATE; decrements balance atomically.
Repository Template (domains/wallet/wallet.repository.ts)	findByIdForUpdate: accepts optional tx; uses .forUpdate() for row locking; decrementBalance on same scope.

DI Container (bootstrap/container.ts)	buildContainer(knex) wires: KnexUnitOfWork → WalletRepo → TransactionRepo → TransactionService → TransactionController.
---------------------------------------	---

2.5 Domain Migration Checklist (Per Sprint)

Sprint 1 — Shared Foundation

■	Add shared/db/unitOfWork.ts
■	Add shared/middleware/validate.ts
■	Add shared/middleware/errorHandler.ts
■	Add shared/errors hierarchy
■	Add shared/middleware/asyncHandler.ts
■	Add DI bootstrap (bootstrap/container.ts)
■	Add request correlation id middleware
■	Add architecture linting rules

Sprint 2 — Transaction Domain (Reference Slice)

■	Create domains/transaction/* files
■	Move transaction endpoints to new controller
■	Move business logic into service
■	Move SQL calls into repository
■	Wrap critical flows in runInTransaction
■	Add idempotency key handling
■	Add success/failure integration tests with rollback verification

Sprint 3 — Wallet Domain

■	Apply same layering pattern
■	Add row lock reads for balance-critical actions
■	Refactor updates to transaction-safe service methods
■	Add insufficient balance tests + race condition tests

Sprint 4 — Order Domain

■	Isolate order use cases in service layer
■	Wrap order + inventory write paths in one transaction
■	Add rollback test when downstream write fails
■	Add domain events/outbox for post-commit actions

Sprint 5 — User/Auth Domain

■	Extract auth controllers/services/repositories
■	Add validation schemas for login/register/reset flows
■	Add integration tests for token/session lifecycle

Sprint 6 — Payment/External Integrations

■	Keep external provider calls outside open DB transaction where possible
■	Add webhook idempotency and replay safety
■	Add retry policy with dead-letter plan

Sprint 7 — Cleanup & Hardening

■	Remove legacy mixed-layer code paths
■	Enforce repository-only DB access convention
■	Track rollback rate, latency, deadlock metrics in dashboards

2.6 Team Backlog Cards

ARCH-01	Introduce UnitOfWork and transaction context abstraction
ARCH-02	Add global validation middleware with typed DTO access
ARCH-03	Add centralized error model + middleware
ARCH-04	Add DI container and refactor one vertical slice
TX-01	Migrate transaction domain and enforce idempotent debit flow
WAL-01	Lock-safe wallet balance operations with rollback tests
ORD-01	Atomic order + inventory update workflow
OBS-01	Add commit/rollback monitoring and structured logs

PART 3
DATABASE MODERNIZATION — MySQL to PostgreSQL

3.1 Observations in sabika-test.sql

MySQL-Specific Patterns ■ MySQL-specific AUTO_INCREMENT on all PKs ■ enum(...) across multiple tables ■ tinyint(1) used as boolean flags ■ datetime + ON UPDATE CURRENT_TIMESTAMP ■ ENGINE=InnoDB + mixed COLLATE directives ■ Duplicate FK definition in gold_transactions dump	Quality Issues to Fix ■ Mixed collations: unicode_ci, general_ci, 0900_ai_ci ■ Inconsistent naming: snake_case mixed with camelCase ■ Financial fields mixed: double, float, decimal ■ 8 business domains — users, auth, wallet, orders ■ inventory, transactions, content, support, notifications ■ Critical: walletBalance stored as double (unsafe for money)
---	---

3.2 PostgreSQL Design Standards

Standard	Rule	Rationale
IDs	bigint generated by default as identity	Safe auto-increment for high-growth tables
Money/Fees	numeric(18,2)	Exact decimal arithmetic — never float or double
Metal/Weight	numeric(18,3)	Domain-appropriate precision
Booleans	boolean	Replace all tinyint(1) flags
JSON Payloads	jsonb	Indexable binary JSON
Timestamps	timestamptz not null default now()	Timezone-aware; replaces datetime
Updated At	Trigger-based update of updated_at	Replaces MySQL ON UPDATE CURRENT_TIMESTAMP
Enums/Status	Constrained text or lookup table FK	Easier to evolve than PG enum types
Soft Delete	deleted_at timestamptz null	Partial index: WHERE deleted_at IS NULL
Naming	snake_case for all identifiers	Rename firstName→first_name, walletBalance→wallet_balance
Text	text (unbounded) or varchar(n) if business rule	Drop arbitrary length constraints

3.3 MySQL → PostgreSQL Type Mapping

MySQL Type	PostgreSQL Type	Notes
int AUTO_INCREMENT	integer generated by default as identity	Use bigint if growth risk
tinyint(1)	boolean	Direct boolean mapping
tinyint (non-bool)	smallint	Non-boolean meaning preserved
double / float (financial)	numeric(p, s)	CRITICAL — exact arithmetic required
datetime	timestamptz	Preferred; timezone-aware

MySQL Type	PostgreSQL Type	Notes
json	jsonb	Binary JSON, indexable
enum('a','b',...)	text + CHECK or FK to lookup table	Easier to evolve
longtext	text	Unbounded text
varchar(n)	varchar(n) or text	Keep if bounded validation is meaningful

3.4 Financial Integrity & Atomic Transaction Design

This is the most critical design area for the Sabeka project. Any operation touching balances, orders, ledger, or inventory must run inside one DB transaction with explicit commit/rollback.

Required Atomic Write Groups - orders + order_details + transactions - transactions + wallet balance + rejection metadata - inventory + inventory_details + fulfillment status - SELECT ... FOR UPDATE on balance/inventory records - Idempotency keys on payment/webhook requests	Isolation & Constraint Controls - Default READ COMMITTED for general operations - REPEATABLE READ or SERIALIZABLE for settlement/reconciliation - Unique constraints on request_id, provider order IDs - Retry policy: bounded retries with backoff on deadlock - Non-negative CHECK constraints on all monetary columns
---	---

3.5 Table-Class Refactor Recommendations

Table / Group	Priority	Key Changes Required
transactions	CRITICAL	Convert walletBalance double→numeric; rename camelCase→snake_case; unique index
orders + order_details	CRITICAL	Convert double→numeric; add non-negative CHECK; index (user_id, created_at)
inventory + inventory_details	HIGH	Normalize status flags; add unique/indexes for serial and stock lookup
users + user_sessions + otp	HIGH	Standardize phone/email unique strategy; add expiry indexes for OTP/session cleanup
gold_transactions	HIGH	Remove duplicate FK ibfk_3/ibfk_4; convert to numeric; rename columns
Translation tables	MEDIUM	Enforce unique (parent_id, locale) pair on every translation table

3.6 Migration Architecture — 6 Phases

Phase	Name	Key Activities	Output
Phase A	Discovery & Freeze	Inventory DDL anomalies; capture slow query logs; define app compatibility matrix	Anomaly catalog + compatibility matrix
Phase B	PG Schema Bootstrap	Create canonical Postgres schema from migration scripts; add constraints, indexes, updated_at triggers	Clean PG schema with all integrity controls
Phase C	Data Migration & Validation	Full load via pgloader; row count + checksum + FK integrity checks; fix rejected records	Validated PG dataset matching MySQL source
Phase D	Dual-Write / CDC Sync	Keep MySQL as source; replicate delta changes to Postgres; shadow reads from app	Shadow comparison reports

Phase	Name	Key Activities	Output
Phase E	Cutover	Read-only window; switch app connections to Postgres; monitor KPIs and rollback triggers	Production on PostgreSQL
Phase F	Stabilization	Keep rollback playbook; tune indexes from real traffic; decommission MySQL path	Sign-off and MySQL decommission

3.7 Tooling & Application Layer Changes

Migration Tooling ■ knex migrations or Prisma for schema management ■ pgloader for first-pass MySQL→Postgres transfer ■ Custom ETL scripts for enum/status renaming ■ camelCase→snake_case column name mapping scripts ■ Row count diff scripts per table ■ Nullability and FK violation check scripts ■ Numeric precision checks on financial columns	Application Layer Changes ■ Replace MySQL-specific SQL functions/syntax ■ Ensure query builders generate Postgres-safe SQL ■ Enforce repository methods accept transaction context ■ Ensure critical service methods wrap writes in one TX ■ Standardize timezone handling (timestamptz round-trip) ■ Keep DTO validation at API boundary ■ Update connection strings and pool configuration
--	--

Sprint-Ready Execution Plan

Sprint	Focus	Key Deliverables
Sprint 1	Audit & Target Schema	Naming/type standards; mapping catalog; critical transaction flow definitions
Sprint 2	Postgres Foundation	PG schema for users, orders, transactions, inventory; conversion scripts; first migration rehe
Sprint 3	Full Schema Migration	Complete schema migration; validate counts, constraints, critical reports; concurrency tests
Sprint 4	App Adaptation	Deploy Postgres-compatible queries; dual read/compare strategy; index tuning
Sprint 5	Production Cutover	Execute cutover runbook; live KPI monitoring; post-cutover hotfix window; final sign-off

APPENDIX

RISK REGISTER & DEFINITION OF DONE

Risk Register

Risk	Level	Mitigation
Financial rounding differences during type conversion	HIGH	Test datasets with known edge financial cases; precision verification scripts
Enum/state mapping mismatch causing business logic drift	HIGH	Explicit state mapping dictionary with automated verification scripts
Hidden MySQL-specific query assumptions in app code	HIGH	Endpoint-level regression tests; replay tests against Postgres
Concurrency behavior changes under different isolation semantics	HIGH	Load testing on transaction-heavy flows before production cutover
Behavior regression during modular refactor	MEDIUM	Incremental migration with feature parity checks; fault-injection rollback tests
Hidden partial update paths in legacy flows	MEDIUM	Fault-injection rollback tests for all critical write flows
Performance regressions due to locking/transactions	MEDIUM	Monitor lock waits and query latency from first rollout
Team inconsistency in applying architecture rules	LOW	Architecture checklist enforced in PR template and CI rules

Definition of Done

DB Modernization Complete When:

✓	Postgres schema follows standardized naming, types, and constraints
✓	Critical financial flows are transaction-safe and rollback-tested
✓	Row counts and integrity checks pass for all migrated tables
✓	Application passes full regression and load testing on Postgres
✓	Observability confirms stable latency/error/rollback KPIs after cutover

Modular Refactor Complete When:

✓	Every migrated domain follows Controller → Service → Repository
✓	Critical write flows use explicit UnitOfWork transactions
✓	Rollback integration tests verify no partial persistence
✓	Validation middleware protects all write endpoints
✓	Error responses standardized through one global handler
✓	Services are dependency-injected and unit-test friendly
✓	Production dashboards include rollback/deadlock/error trend visibility

System Design Adoption Complete When:

✓	Every domain has clear boundaries and ownership
✓	Service layer is deterministic, testable, and transaction-safe
✓	Data consistency guaranteed by transaction and constraint design
✓	APIs are validated, secure, and operationally observable
✓	Scaling decisions (workers, replicas, caching) supported by architecture

Immediate Next Actions

Action 1	Build full MySQL→Postgres column mapping sheet from sabika-test.sql
Action 2	Freeze naming/type conventions and publish as architecture rules
Action 3	Prototype migration for 4 critical tables: users, orders, transactions, inventory
Action 4	Add transaction-focused integration tests before production migration
Action 5	Introduce UnitOfWork abstraction and migrate transaction domain as reference slice
Action 6	Schedule two dry-run rehearsals before final DB cutover
Action 7	Set up SLO dashboards for rollback rate, deadlock count, transaction latency

Prepared for **Sabeka** backend architecture, database modernization, and modular refactor planning. Document generated on April 29, 2026.